

Development & Engineering Guidelines

Evora — Internal Standards Document

Evora is building a connected digital ecosystem around premium electric vehicles: direct-to-consumer sales, subscription-based ownership, smart-fleet analytics for enterprise clients, and a global network of branded showrooms. Our engineering standards exist to support that vision — a unified platform, end-to-end digital customer experience, real-time insight for showroom teams, and AI-driven lead assignment and forecasting — without compromising the trust customers place in us during high-value purchases.

This document brings together the standards, principles, and ways of working that apply to teams building applications across Evora. It has evolved with input from Security, Architecture, Compliance, and the broader engineering community, and applies across all our operating regions (EU, UK, UAE, and Singapore). Teams are expected to be familiar with the contents and to apply good judgment when interpreting them for their own projects. Where something is unclear, reach out to the relevant function.

Throughout this document, rules are grouped into **Required** (hard rules that must be followed) and **Recommended** (softer expectations to apply with judgment).

Security

Security is the area where mistakes are most expensive and hardest to undo, so it carries the largest set of hard rules in this document. Evora handles high-value vehicle purchases, recurring subscription payments, and personal data tied to identifiable customers and their vehicles — a credential leak or access-control gap affects customer trust, our brand, regulatory standing, and the wider connected platform. Several of the rules below exist precisely because access-control mistakes are easy to make in multi-tenant and anonymous-user scenarios, such as public-facing showroom and configurator experiences. The expectation is that security is designed in from the start rather than bolted on before deployment. The rules cover password handling, authentication, anonymous users, and deployment hygiene; when in doubt about how strictly any of them applies to your project, treat the stricter interpretation as the default and confirm with the Security function.

Passwords & hashing

- Never store passwords in plain text under any circumstances.
- Do not use MD5 or SHA1 hashing. Use BCrypt (preferred); SSHA256 is also acceptable.
- Enforce a minimum password length of 14+ characters with standard complexity expectations.
- Rotate passwords roughly every 90 days. (*Note: industry best practice on rotation is under debate.*)

Authentication & access

- Tier A applications (business-critical or externally-facing, e.g. customer accounts, subscription management, fleet portals) **must** use Single Sign-On via the company SAML provider — this is not optional.
- Multi-tenant applications (e.g. enterprise fleet clients sharing a platform) must use the `CurrentUser` construct in their access rules.
- If a user role has more than ~10 module roles attached, flag it for security review (not strictly forbidden).

Anonymous users

- Minimize the use of anonymous users; prefer authenticated users wherever possible. Public

experiences such as the vehicle configurator and test-drive booking are common exceptions — scope their access tightly.

- Anonymous users must **not** be able to create persistent entities.
- Anonymous users must **not** edit string attributes with no length limit (attack vector).
- Entity access rules must be actually applied in document generation flows (e.g. quotes, contracts, invoices).

Deployment & secrets

- Set Project Security to Production mode before deployment, with a check in place to confirm it.
- Never place credentials of any kind (API keys, passwords, connection strings, tokens) in code, configuration files, or logs — including temporary debug logging.

Sessions & access rules (recommended)

- Use shorter session timeouts (~15 minutes) for sensitive-data applications such as payment and subscription management; ~60 minutes is acceptable for standard internal applications.
- Consider read-only XPath constraints in entity access rules where applicable.

Architecture

Our architectural philosophy leans toward domain-driven design, with the depth of application scaling to project complexity — an internal showroom tool does not need the same rigor as the customer-facing subscription platform or the smart-fleet analytics engine, but both benefit from clear domain boundaries and well-structured logic. The goal is applications that are maintainable as teams change, performant under real load, and able to scale horizontally as Evora grows across regions. Most of the hard rules here address failure modes known to cause production problems: blocking the UI on slow operations, committing inside loops, and recursive microflows all create issues that are painful to diagnose after the fact — and that matters when showroom teams depend on real-time responsiveness. The recommended patterns, by contrast, are tools to reach for when the problem calls for them rather than defaults to apply everywhere; the right choice depends on the domain, the read/write profile, and how loosely coupled the system needs to be. Favor the simplest design that meets the requirement, and add architectural complexity only when it earns its place.

Required

- Follow domain-driven design principles (depth depends on project complexity).
- Include appropriate error handling and logging in all microflows — errors caught, logged with enough detail to diagnose, and handled gracefully for the user.
- Use asynchronous processing for long-running operations (rough threshold: anything over ~5 seconds) rather than blocking the UI.
- Do not commit to the database inside loops (causes performance and transactional issues).
- Do not use recursive microflows (microflows that call themselves).

Recommended

- Design for horizontal scalability, which generally means stateless applications.
- Consider microservices architectures for more complex domains (e.g. separating fleet analytics from retail sales).
- Consider CQRS patterns for read-heavy applications such as real-time showroom dashboards and forecasting.
- Consider event-driven architectures for loosely coupled systems (e.g. reacting to vehicle telemetry or lead events).

- Cache frequently accessed reference data (e.g. vehicle models, configuration options, pricing tables).
 - Keep microflow nesting to ~3 levels or fewer; beyond that becomes hard to maintain.
-

Compliance — Data Privacy, Financial, and Related Areas

Evora operates across the EU, UK, UAE, and Singapore, which means compliance is not a single checklist but a set of overlapping obligations that depend on where an application operates and what data it touches — GDPR (EU) and UK GDPR are primary, with the UAE's PDPL and Singapore's PDPA also in scope, alongside PCI-DSS for payment handling. The points below capture the most important and most commonly applicable requirements, but they are not exhaustive; a given product may carry additional obligations, and teams should treat this section as a floor rather than a ceiling. Two themes run through everything here: we collect and retain the minimum personal data necessary, and we maintain the audit trails needed to demonstrate compliance after the fact. Many of these requirements (automated retention, logging of PII access, segregation of duties) cannot be retrofitted easily, so they need to be designed into the data model and workflows early. When a new feature will process personal, vehicle, or payment data, involve the Compliance function before building rather than after.

Data privacy — required

- Collect only PII that is required for the service being delivered (customer, subscription, and vehicle-linked data).
- Implement the “right to erasure” (right to be forgotten) for all applications handling personal data.
- Provide data portability through export functionality.
- Apply retention policies that automatically delete data after the retention period expires (not handled manually after the fact).
- Encrypt PII both at rest and in transit.
- Obtain explicit consent before processing sensitive personal data.
- Log all PII access for audit purposes — non-negotiable.

Data privacy — recommended

- Pseudonymize PII where it makes sense (e.g. in analytics and forecasting datasets).
- Conduct privacy impact assessments when new features will process personal data.

Financial — required

- Enforce segregation of duties — no single user controls a financial transaction or process (e.g. a vehicle sale or subscription change) end-to-end.
- Maintain audit trails for all financial transactions in a way that supports later review.
- Never store credit card numbers, CVV codes, or full account numbers; use tokenization for payment processing.
- Apply end-to-end encryption to financial data.

Financial — recommended

- Use approval workflows for transactions above defined thresholds (relevant for high-value vehicle purchases and enterprise fleet contracts).
 - Reconcile daily with source systems. (*Exact thresholds and frequencies depend on application context.*)
-

API Design and Integration

REST APIs are our primary integration approach across Evora, and because they connect our show-room systems, subscription platform, payment providers, and fleet-analytics services to each other and to external partners, consistency matters as much as correctness — an API that behaves predictably is far easier to consume, version, and secure. The conventions below exist so that a developer moving between teams encounters the same patterns for versioning, HTTP semantics, and status codes rather than relearning each service's quirks. The security rules are non-negotiable: APIs are a primary attack surface, and gaps in input validation, escaping, or rate limiting are a direct risk to customer and payment data. The documentation and design recommendations help consumers — including enterprise fleet clients — integrate quickly and safely; apply them wherever they add value, and always err on the side of clear, well-documented contracts over clever ones.

Conventions

- Version in the URL path (e.g. `/api/v1/`, `/api/v2/`).
- Use HTTP methods semantically: GET for reads, POST for creates, PUT/PATCH for updates, DELETE for deletes.
- Return status codes that reflect the actual outcome — do not return 200 with an error in the body.

Security — required

- Apply rate limiting to protect against denial-of-service.
- Validate and sanitize all API inputs.
- Properly escape REST call templates to prevent injection attacks.
- Use OAuth 2.0 for authentication (JWT-based authentication is acceptable in some cases).

Documentation & design — recommended

- Paginate list endpoints, with a typical maximum page size of 100 items.
- Use HATEOAS (links to related resources) where it adds value.
- Document APIs using OpenAPI/Swagger — especially partner- and fleet-facing APIs.
- Log requests/responses for debugging, but never capture sensitive payload contents.

Code Quality and Naming

Consistent naming and quality standards are what make a codebase legible to people who did not write it — and as Evora's platform grows and applications outlive the teams that build them, that legibility is a real cost saver. Naming conventions come up frequently in code reviews, often as friction that could have been avoided, so they are listed explicitly below rather than left to individual preference. The aim is that anyone opening a project can infer what a microflow does from its name, tell a constant from a variable at a glance, and know which module an entity belongs to. The quality rules separate hard correctness and security concerns — null pointer risks, exposed constants, retracted Mendix versions — from the softer craft habits that keep code healthy over time, such as keeping methods small and letting clear names do the work that comments otherwise would.

Naming conventions

- PascalCase for entities, modules, pages, and microflows.
- camelCase for attributes, variables, and parameters.
- UPPER_SNAKE_CASE for constants.
- Use descriptive names; avoid abbreviations unless very widely understood (e.g. ID, URL, EV, VIN).
- Microflow type prefixes: ACT_ (action), GET_ (retrieval), SUB_ (submicroflow).

- Microflow names follow a verb-noun pattern (e.g. CreateOrder, BookTestDrive, AssignLead).
- Shared entities carry a module prefix (e.g. SALES_Order, FLEET_Vehicle, SUB_Subscription).

Quality — required

- Avoid empty variables that would cause null pointer exceptions in actions.
- Never expose sensitive constants to the client side.
- Handle all error scenarios with appropriate user-facing messages.
- Do not use retracted Mendix versions in any project.

Quality — recommended

- Prefer self-documenting code (clear names, simple logic) over explanatory comments; reserve comments for genuinely complex business logic.
 - Refactor methods that grow beyond ~50 lines.
 - Maintain consistent formatting and indentation (linting is in place to help).
 - Conduct peer code review before merging.
-

Dependencies and Marketplace Modules

Every external module or dependency we pull into a project is code we did not write but are nonetheless responsible for — it can carry security vulnerabilities, licensing obligations, or quality issues that become our problem in production. Given the customer and payment data flowing through Evora's platform, an unvetted dependency is a genuine supply-chain risk, which is why Marketplace modules and external dependencies go through a review process before use rather than being adopted ad hoc. The approved list exists to give teams a fast, safe default: if a module is on it, the security, licensing, and quality review has already been done. The required rules ensure nothing slips in unreviewed and that we can account for what we depend on during audits. The recommendations reflect good long-term hygiene — preferring internally maintained modules, keeping dependencies current, and resisting the gradual accumulation of libraries that each add surface area and maintenance burden.

Required

- Use only Marketplace modules from the approved list; modules outside it require review before use.
- Scan external dependencies (including those bundled in Marketplace modules) for security vulnerabilities.
- Do not introduce dependencies with known critical or high severity issues.
- Document all external dependencies along with their purpose (supports later audits).

Recommended

- Prefer internal Evora marketplace modules over public ones.
 - Review and update dependencies quarterly.
 - Minimize the overall number of dependencies.
-

Accessibility

Accessibility is both a legal obligation and a matter of building applications that everyone can actually use, and we hold ourselves to WCAG 2.1 Level AA as our standard — fitting for a brand whose customer experience is meant to be delightful to all. Treating accessibility as a first-class requirement rather than a late-stage audit produces better software for every user, not only those relying

on assistive technology — keyboard navigation, clear semantics, and sufficient contrast benefit everyone exploring vehicles online or in a showroom. The required rules below represent the baseline we commit to on every application: they are not optional polish. The recommended practices go a step further toward genuinely good accessibility, most notably testing with real assistive technologies rather than trusting automated checkers alone, which catch only a fraction of real-world issues. Design for accessibility from the outset; retrofitting it onto a finished interface is consistently more expensive and less effective than building it in.

Required

- Ensure keyboard navigation works for all interactive elements.
- Apply ARIA labels and semantic HTML properly.
- Provide screen reader support.

Recommended

- Test with actual assistive technologies, not just automated tooling.
- Provide text alternatives for images (e.g. vehicle photography and configurator visuals).
- Meet a color contrast ratio of at least 4.5:1.
- Avoid content that flashes or strobos (seizure risk).